



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Department of Physics and Astronomy "A. Righi"

Theoretical Physics

Complex Networks

Lectures held by
Prof. Daniel Remondini

Notes by
Gioele Mancino

Academic Year 2025/2026

Contents

1	Definitions	3
1.1	Spectral Network Analysis	6
	Appendices	11
A	Notation and Conventions	11
B	Computations and further developments	11

Introduction

1 | Definitions

Star Network

The star network is a simple network topology where one central node is connected to all other nodes, which are not connected to each other. This structure resembles a star, with the central node acting as the hub and the other nodes as the spokes. The star network is often used in communication systems and computer networks due to its simplicity and ease of management. However, it has a single point of failure, as the central node's failure can disrupt the entire network.

Tree Network

A tree network is a hierarchical structure that consists of nodes connected in a parent-child relationship. It is a type of acyclic graph where each node has one parent and can have multiple children. The topmost node is called the root, and the nodes at the bottom are called leaves. Tree networks are commonly used in organizational structures, file systems, and decision-making processes. They allow for efficient data organization and retrieval but can become inefficient if the tree becomes too deep or unbalanced.

It is characterized by its branching structure, where each node can have multiple child nodes, but only one parent node. This structure allows for a single path between any two nodes, making it easy to navigate and manage, there is no redundancy in the connections, which can lead to efficient communication and data transfer. However, it can also lead to bottlenecks if the root node or any intermediate nodes fail, as it can disrupt the entire network. With multiple tree networks, it is possible to create a more complex structure, such as a **forest**, which consists of multiple trees that are not connected to each other. This can provide redundancy and improve the overall resilience of the network.

Fractal as a Regular Tree [...]

1.0.1 | Directed Acyclic Graph (DAG)

A directed acyclic graph (DAG) is a type of graph that consists of nodes connected by directed edges, where there are no cycles. In a DAG, each edge has a direction, indicating the flow of information or dependencies between nodes. DAGs are commonly used in various applications, such as scheduling tasks, representing dependencies in software development, and modeling hierarchical relationships. It can be used to represent complex systems where there are multiple levels of dependencies, without the risk of creating cycles that can lead to infinite loops or deadlocks.

With DAGs we can represent more complex relationships than trees, as they allow for multiple paths between nodes, establishing an effective hierarchy. However, they can also be more difficult to manage and navigate, especially as the number of nodes and edges increases. DAGs are often used in scenarios where there are multiple dependencies or relationships between entities, such as in project management, data processing pipelines, and knowledge representation.

For networks with these properties, the **adjacency matrix** is upper triangular, which means that all the entries below the main diagonal are zero. This is because in a DAG, there are no cycles, and therefore, there can be no edges that point back to previous nodes. The upper triangular structure of the adjacency matrix reflects the hierarchical nature of the DAG, where nodes can only have edges pointing to nodes that come after them in the hierarchy. Through this matrix, we can represent the structure of the DAG and analyze its properties as a linear algebra problem, such as finding the longest path or determining the reachability of nodes.

Min/Max Spanning tree MST

A minimum spanning tree (MST) is a subset of the edges of a connected, weighted graph that connects all the vertices together without any cycles and with the minimum possible total edge weight. It is a method for finding the most efficient way to connect all the nodes in a graph while minimizing the total cost: it finds the backbone of the network. The MST can be used in various applications, such as designing efficient communication networks, clustering data, and solving optimization problems. There are several algorithms for finding the MST, including Prim's algorithm and Kruskal's algorithm, which are based on greedy strategies to build the tree incrementally by selecting the edges with the smallest weights.

Kruskal's Algorithm Kruskal's algorithm is a popular method for finding the minimum spanning tree (MST) of a connected, weighted graph. The algorithm works by sorting all the edges of the graph in non-decreasing order of their weights and then adding edges to the MST one by one, as long as they do not form a cycle. The process continues until there are $V - 1$ edges in the MST, where V is the number of vertices in the graph. To resume:

1. Start: single node and list of all links (edges);
2. Sort all edges in non-decreasing order of their weights;
3. Add edges to the MST one by one, ensuring no cycles are formed;
4. Continue until there are $V - 1$ edges in the MST, where V is the number of vertices.

Kruskal's algorithm is efficient and can be implemented using a disjoint-set data structure to keep track of the connected components of the graph, ensuring that cycles are not formed when adding edges to the MST.

Prim's Algorithm [...]

Example (Pearson Correlation Coefficient). Pearson's correlation matrix (distance $d = 1 - r$, $r > 0$, $d \in [0, 2]$) is a common way to represent the relationships between nodes in a network, where the correlation coefficient r measures the strength and direction of the linear relationship between two variables. The distance d is calculated as $d = 1 - r$, which transforms the correlation

coefficient into a distance metric that can be used to analyze the network structure. In this representation, a high correlation (close to 1) corresponds to a small distance (close to 0), indicating that the nodes are closely related, while a low correlation (close to -1) corresponds to a large distance (close to 2), indicating that the nodes are not closely related.

It is a fully connected weighed network, which is very difficult to analyze and visualize. Connectivity can be reduced by considering its MST: if we find high correlation between two nodes, we can consider them as connected, otherwise not (correlation is a measure of similarity between two nodes: when the correlation is high, the nodes are similar as $A \equiv B$, $B \equiv C$, then $A \equiv C$ if A and C are also highly correlated). Doing this, we can find the backbone of the network, which is a tree structure that captures the most important relationships between the nodes. This transformation allows us to analyze and visualize the network more effectively, as it reduces the complexity while preserving the essential connections. However, it is important to note that this approach lead to the **loss of some information**, as it removes weaker connections that may still be relevant in certain contexts.

LIMITS: ALL cycles are removed: very drastic transformation

Global rules for link selection We can also consider other global rules for link selection, which involve setting a threshold to determine which links to keep in the network. For example:

- consider the total distribution of link weights W and define a cutoff threshold C to keep only $W_{ij} > C$;
- define a parametric link distribution (eg Gaussian, Gamma) and keep only links so that $p(W_{ij}) < p_0$.

But this approach has some limitations, as it can lead to the loss of important information about the network structure:

- Global thresholding methods miss the local relevance of low link values (different weight distribution as a function of local node features such as their degree).

Local rules for link selection Local methods analyze weights in node neighborhoods Link selection based on single-node null model (pdf)

Example (Serrano Boguna Vespignani PNAS07).

Node of degree K with positive weighted links W_{ij} (normalized to 1, can use absolute value if also negative) Hypothesis: weights are extracted from a random split of unit segment $[0:1]$ in K parts (geometric distribution $G(K)$) Keep only links for which $G(W_{ij}) < p_0$ (global threshold but applied to local node-dependent distribution)

1.1 | Spectral Network Analysis

Through the analysis of the eigenvalues and eigenvectors of matrices associated with a network, such as the adjacency matrix or the Laplacian matrix, we can gain insights into the structural properties of the network. Spectral analysis can reveal important information about the connectivity, community structure, and dynamics of the network. For example, the largest eigenvalue of the adjacency matrix can indicate the presence of hubs in the network, while the second smallest eigenvalue of the Laplacian matrix can provide information about the connectivity and robustness of the network. Spectral methods are widely used in various applications, including clustering, dimensionality reduction, and network visualization.

We can read the problem as an **eigenvalue problem**:

$$A \cdot \mathbf{x} = \lambda \cdot \mathbf{x},$$

where A is the adjacency matrix of the network, $\mathbf{x}$ is an eigenvector, and λ is the corresponding eigenvalue. The eigenvalues and eigenvectors can provide insights into the structure and properties of the network, such as identifying communities, detecting central nodes, and understanding the dynamics of processes on the network. Spectral analysis can also be used to identify important features of the network, such as its connectivity, robustness, and resilience to perturbations.

Example (Principal Component Analysis (PCA)). Principal Component Analysis (PCA) is a dimensionality reduction technique that can be applied to network data to identify the most important features or components that capture the variance in the data. By analyzing the eigenvalues and eigenvectors of the covariance matrix of the network data, PCA can help to identify patterns and relationships between nodes, as well as reduce the complexity of the data while preserving its essential structure. PCA can be particularly useful for visualizing high-dimensional network data and for identifying clusters or communities within the network.

We can also imply the **characteristic polynomial** of the adjacency matrix A , defined as

$$\Phi(\lambda) = \det(A - \lambda I),$$

where I is the identity matrix. The roots of this polynomial are the eigenvalues of A , and analyzing them can provide further insights into the structural properties of the network.

1.1.1 | Power of A and Shortest Path

Let us start by defining the powers of the adjacency matrix A . The n -th power of the adjacency matrix, denoted as A^n , can be computed using matrix multiplication. The entry $(A^n)_{ij}$ represents the number of paths of length n from node i to node j in the network:

$$B = A^2, \quad B_{ij} = \sum_k A_{ik} A_{kj}, \quad B_{ii} = \sum_k A_{ik} A_{ki}.$$

This is because each multiplication of the adjacency matrix counts the number of ways to traverse from one node to another through intermediate nodes. This multiplication tells us how many paths of length n exist between nodes i and j . For example, if $A_{ij}^2 > 0$, it means there is at least one path of length 2 (since we are looking at the second power) connecting nodes i and j .

B_{ii} can infer on directed and undirected paths in the network, as it counts the number of paths that start and end at the same node i . In an **undirected network**, this would count the number

of loops or cycles that involve node i , while in a **directed network**, it would count the number of directed paths that return to node i . This information can be useful for understanding the local structure and connectivity of the network around node i .

- The first non-null term ij of the k -th power of A indicates the length of the shortest path from i to j (without specifying the route(s)).
- The k -th power matrix counts the number of walks from one node to another.
- If an element $A_{ij}^k = 0$ for each k , there are no paths between the two nodes (up to $k_{\max} = \# \text{nodes}$).
- If the graph is directed, certain paths/walks may not exist.
- If no paths exist between two nodes in a symmetric graph, then the network is disconnected.

Cycles A *directed cycle* is a path that starts and ends at the same node, following the direction of the edges

$$(C_{ND})^N = \mathbb{I} \implies \lambda_i^N = 1 \implies \lambda_i = e^{2\pi i k/N} \quad \text{for } k = 0, 1, \dots, N-1,$$

showing that the eigenvalues of the adjacency matrix of a directed cycle graph are the N -th roots of unity. This means that the eigenvalues lie on the unit circle in the complex plane, and their distribution can provide insights into the structure and properties of the directed cycle graph. The presence of cycles in a network can have significant implications for its dynamics and stability, as they can lead to feedback loops and complex interactions between nodes. For example, if $N = 4$ we have:

$$C_{4D} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

[...]

Completely connected graphs These graphs have a simple structure, where every node is connected to every other node. The adjacency matrix of a completely connected graph with N nodes can be represented as:

all-ones matrix, rank 1, with eigenvalues N and 0 (with multiplicity $N-1$)

If we remove the self-loops (i.e., the diagonal entries), we get a traceless matrix with $N-1$ entries equal to 1 in each row, which can be represented as:

$$A = \begin{pmatrix} 0 & 1 & 1 & \cdots & 1 \\ 1 & 0 & 1 & \cdots & 1 \\ 1 & 1 & 0 & \cdots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 0 \end{pmatrix},$$

which is traceless and has $N-1$ entries equal to 1 in each row (since each node is connected to $N-1$ other nodes). The diagonal entries are zero because there are no self-loops in the graph.

This structure leads to a highly symmetric adjacency matrix, which has implications for its spectral properties and the overall connectivity of the network.

The eigenvalues of this adjacency matrix can be calculated, and they consist of one eigenvalue equal to $N - 1$ (which corresponds to the eigenvector with all entries equal) and $N - 1$ eigenvalues equal to -1 (which correspond to the eigenvectors that are orthogonal to the all-ones vector). This spectral property reflects the high level of connectivity in the completely connected graph, where each node is directly connected to every other node. The presence of a single large eigenvalue indicates that there is a strong overall connection in the network, while the negative eigenvalues indicate that there are no other significant structures or communities within the network.

Bipartite graphs

DAG

Note on Cycles and Network Sparsity

If a network has no cycles there is a single path between any two nodes (which is the shortest path necessarily): it is a powerful approximation for many real networks, which are often very sparse (e.g. with a small number of links as compared to the number of nodes $o(N^2)$). If a network is sufficiently sparse (e.g. with a small number of links as compared to the number of nodes $o(N^2)$) it is very likely not to have cycles, or they can be considered negligible (e.g. in generating multiple paths between nodes): we can consider the network as a **tree**, or more generally as a DAG (if directed), without cycles.

This is an approximation used very often in theorems, in particular for random networks ensembles (see following lectures) In sparse networks cycles are rare, and long cycles even more.

Thus the presence of cycles is an indicator of a particular network design (that cannot be obtained by randomly connecting nodes, see following lectures on network entropy)

What happens to the spectrum of these matrices if we perturb slightly the network structure? For example, if we add or remove a few edges or loops (ciclicity, bipartition, all-ones...), how does this affect the eigenvalues and eigenvectors of the adjacency matrix? This question is important for understanding the stability and robustness of the network, as well as for analyzing the dynamics of processes that occur on the network. Perturbations can lead to changes in the spectral properties of the network, which can in turn affect its overall behavior and functionality. They can be studied by analyzing the sensitivity of the eigenvalues and eigenvectors to changes in the adjacency matrix, generating random matrices and analyzing their spectra, or by using perturbation theory to understand how small changes in the network structure can lead to changes in its spectral properties (like adding random links or random weights to the adjacency matrix). This analysis can provide insights into the resilience of the network to perturbations and its ability to maintain its functionality under changing conditions.

1.1.2 | Isomorphism - Spectra

Two graphs are isomorphic if they are equal up to node permutation (transformation of similitude in matrix algebra)

$$A_1 = P A_2 P^T, \quad (P P^T = \mathbb{I})$$

$$|A_1| = |P A_2 P^T| \implies |A_1| = |A_2|,$$

thus isomorphic graphs have the same spectrum, but the converse is not necessarily true: two non-isomorphic graphs can have the same spectrum (called **cospectral** graphs):

$$\text{Isomorphic} \implies \text{Cospectral},$$

$$\text{Non-Cospectral} \implies \text{Non-Isomorphic}.$$

This means that while the spectrum of a graph can provide important information about its structure and properties, it is not a complete invariant for graph isomorphism. Therefore, additional methods or invariants may be needed to determine whether two graphs are isomorphic or not, especially in cases where they are cospectral.

But co-spectral DOES NOT imply isomorphic! (for $N > 4$ knots) [Can one hear the shape of a drum? M. Kac Am Month. 73 (4), 1966]

Let us resume some of the key points about the spectrum of a graph:

- Eigenvalue spectrum: "Summary", "modes" of a graph, tells us about the structure of the graph, but not all the information (e.g. it does not tell us about the specific arrangement of nodes and edges, or about the presence of certain substructures or motifs in the graph)
- Spectrum is independent of representation (invariant for node re-indexing)
- It is invariant for permutation operation: being a similarity transform (involving a matrix and its inverse) the eigenvalue problem is identical (related to the properties of the determinant of matrix products) problem is identical (related to the properties of the determinant of matrix products)
- There are few exact results and some numerical results for particular cases (more on trees, less on general graphs)

Co-spectrality

1.1.3 | Graph Spectral Distance

Once the eigenvalues have been sorted (with the addition of zeros for networks with different number of nodes, corresponding to isolated nodes that do not contribute to spectrum calculation)

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N,$$

$$\mu_1 \geq \mu_2 \geq \dots \geq \mu_N, d(G_1, G_2) = \sqrt{\sum_{i=1}^N (\lambda_i - \mu_i)^2},$$

we can define a spectral distance between two graphs G_1 and G_2 as the Euclidean distance between their sorted eigenvalue spectra. This distance provides a measure of how similar or different the two graphs are in terms of their spectral properties. A smaller distance indicates that the graphs have

more similar spectral characteristics, while a larger distance indicates that they are more different. This spectral distance can be used for various applications, such as clustering graphs, comparing network structures, and analyzing the evolution of networks over time. However, it is important to note that this distance is not a complete invariant for graph isomorphism, as two non-isomorphic graphs can have the same spectrum (co-spectral graphs). Therefore, while the spectral distance can provide useful insights into the similarity of graphs, it should be used in conjunction with other methods for a more comprehensive analysis of graph structure and properties.

1.1.4 | Similar Graphs and Their Spectra

To define similar graphs we have to define a distance between graphs, which can be done using the spectral distance defined above. Similar graphs are those that have a small spectral distance, indicating that their eigenvalue spectra are close to each other. This means that the structural properties of the graphs are similar, as reflected in their spectra. Similar graphs may have similar connectivity patterns, community structures, or other structural features that contribute to their spectral similarity. However, it is important to note that similar graphs may not necessarily be isomorphic, as they can have different arrangements of nodes and edges while still having similar spectral properties. Therefore, while the spectral distance can provide insights into the similarity of graphs, it should be used in conjunction with other methods for a more comprehensive analysis of graph structure and properties.

There seems to be a strong correlation between spectral distance and graph edit distance. Thus proximity of spectra reflects graphs with similar structure (even if exceptions exist)

Graph edit distance Graph edit distance is a generalization of edit distance between strings of symbols, which is used in bioinformatics or in text analysis to identify similarity in different strings (protein or DNA sequences, words or periods). In symbol sequences, we can have elementary operations like symbol substitution, symbol insertion or symbol removal.

The set of operations that connect one string to the other can be represented in a graph table, with a path representing the editing steps. Different scores can be defined to identify the best matching between two strings (and thus between two graphs), for example

- Longest common subsequence (subgraph)
- Global alignment scores based on different values of penalties associated to mismatches, insertions, deletions

Example (DNA sequences). For proteins and DNA sequences there is a natural node ordering, thus edit distance is simpler to calculate (substitution/insertion/deletion along the sequences). For networks there is no natural node ordering, thus also the starting point of node-to-node mapping is nontrivial. Possible approaches (open problems):

- Spectral approaches to matrices (eigenvectors components and node similarities)
- Similarity between node embeddings

TODO: insert
graphs

Appendices

A | Notation and Conventions

B | Computations and further developments